Making good sorts go bad

2012-07-31 13:58:09

Original: `https://nicknash.me/2012/07/31/adversaries/`

0.1 Or, how can we construct bad inputs for any sorting or selection algorithm?

1 Prologue

Everyone loves sorting. Even people who don't know they do, because our phones, tablets and laptops are always sorting. Keeping things in order is just part of life, and especially so if you're a computer. As each internet packet boinks through the routers of the Internet, its IP address is compared in an ordered way. So these days little bits of sorting are happening almost constantly. As rain falls, sun shines and humanity breathes: computers sort.

2 Adversaries

A while ago, I needed to stress test a sorting-like algorithm. It turned out I needed something a bit smarter than say, McIlroy's (`http://www.cs.dartmouth.edu/~doug/mdmspe.pdf`) quicksort killer. So I came up with the naive (and somewhat impractical) algorithm described below. It's just a bit of idle curiosity, but I hope it's interesting. I'll also present a minor "application" of the algorithm.

So: sorting algorithms sort data. Here, we're going to enter the mind of the data. We're also going to be bad. We're going to think: As a bunch of data, how can I make the job of the sorting algorithm as hard as possible?

3 Killing Quicksort

We all remember quicksort only works in $\Theta(n \log n)$ time on average. Its worst case is quadratic. Yikes.

What's more, the famous Doug McIlroy described (`http://www.cs.dartmouth.edu/~doug/mdmspe.pdf`) a simple way of making pretty much any practical quicksort implementation go quadratic.

In C, we sort things by saying

```
qsort (values, num_values, sizeof(int), compare);
```

McIlroy's quicksort killer lives inside the compare function. It never *lies*, but it doesn't have anything else in mind other than to make quicksort's job difficult. To do this, it uses a simple heuristic to guess which element is the pivot, and assigns it the smallest possible value consistent with the

results of previous calls to compare. The result is that any quicksort examining only a constant number of elements to determine its pivot will go quadratic, *always*.

4 Inside the Killer

The killer works by regarding the entire array to be sorted as *gas* to begin with. An element of the array qualifies as gas if the sorting algorithm hasn't asked enough questions about it yet to force the killer to reveal the element's position in the sorted output. As the sorting algorithm asks questions about the input (by calling compare) , it pins down where some elements belong in the output. The killer keeps track of this by *freezing* these elements into *solid* values. It always freezes gas into the smallest consistent solid value available.

The rules for returning results from the compare function are then simple, if we're comparing:

- Two solid values, just compare them, the killer has already divulged their relative order.
- Solid to gas, the gas is always larger (when the killer freezes the gas later, it'll be consistent about this)
- Gas to gas, freeze one of them with a value larger than any currently frozen value (i.e. `num_solid++`)

The final detail is that, in gas to gas comparisons, we'll freeze the one we think is the pivot. Since we know the partition step repeatedly compares elements to the pivot, we will freeze the gas element that has been involved in the previous comparison (tie-breaking with one or the other if neither has). This matches the goal of trying to freeze the pivot as soon as possible, and thus forcing it to be as small as possible.

McIlroy's code is here (`http://www.cs.dartmouth.edu/~doug/aqsort.c`).

5 Keeping Secrets

McIlroy's program is pretty neat, but it's specific to quicksort. It's imagining those pivot comparisons to decide what to freeze. Can we come up with something that'll work against any old sorting algorithm, or sorting-like algorithm?

Let's think about it this way – we're the space of all possible input arrays. From the sorting algorithm's point of view, we're some particular array $x[1 \ldots n]$. The sorting algorithm is going to ask us questions, like "Is $x[1] < x[2]$?" . Here's the key: Each time we answer, we want to give out as little information as possible.

6 Secret Descendants

Let's get down to business. We can represent the information the sorting algorithm has extracted about the input $x[1 \ldots n]$ as a directed acyclic graph (DAG) on n nodes, labelled $V_1, \ldots, V_n$.

Initially, the DAG has no edges. An edge from V_i to V_j means that $x[i] < x[j]$. So if the sorting algorithm asks us "Is $x[i] < x[j]$?", we either say "Yes", and add an edge from V_i to V_j in the DAG, or say "No", and add an edge from V_j to V_i.

Now imagine V_i has 10,000 descendants, and V_j has only 1. If the sorting algorithm asks "Is $x[i] < x[j]$?" we had better say a loud "No". Answering "Yes" tells it that $x[j]$ is greater than $x[i]$ as well as the 10,000 elements corresponding to the descendants of V_i. Instead, answering "No", tells the sorting algorithm about the relative order of just 3 elements.

7 DAG in Pictures

Let's look at a few pictures of what this looks like. Suppose the DAG adversary is being quizzed about an array $x[1 \ldots 7]$, and it has already answered a few times, giving:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv1.png`

Link: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv1.png`

Now it gets asked "Is $x[1] < x[6]$?". If it says "Yes" we are left with this DAG

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv2.png`

Link: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv2.png`

Whereas answering "No" gives this one:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv3.png`

Link: `http://nicknash.me/wp-content/uploads/2012/07/dag_adv3.png`

Saying "No" told the sorting algorithm more than we had to. It got to know not only that $x[6] \geq x[1]$, but that $x[6]$ isn't less than any of $x[1]$'s descendants.

8 Rules of Secret Descendants

So, let's be precise about how our DAG adversary should work:

Each time the sorting algorithm asks "Is $x[i] < x[j]$?", it answers as follows:

1. If there is a directed path from V_i to V_j , it says "Yes."
2. Otherwise, if there is a directed path from V_j to V_i , it says "No."
3. Otherwise, if V_i has fewer descendants than V_j , it says "No.", and adds an edge from V_j to V_i.
4. Otherwise (V_j has no more descendants than V_i) it says "Yes." and adds an edge from V_i to V_j.

The first two rules just maintain consistency with previous answers the DAG adversary has given. The last two try and leak as little information as possible, just as was described earlier.

9 Does it Work?

If we implement (`https://github.com/nicknash/AdversaryExperiments/blob/master/Adversaries/DAGAdver`
the DAG adversary and give it a whirl against, say C#'s List(T).Sort function, here's what happens:

Success! It certainly looks like we've hurt the C# library function — the red-line is the number of comparisons against input size. The green-line is a guessed at quadratic matching the measured number of comparisons. This is kind of nice, because the adversary knows nothing in particular about the sorting algorithm's implementation, unlike McIlroy's nifty quicksort killer.

10 Stepping Back

What's really going on with this DAG? At any instant, it represents the information the sorting algorithm has extracted about the input. What's more, any topological sorting of the DAG (i.e. a listing of the vertices such that V_i always comes before V_j if there is an edge from V_i to V_j) is an ordering of $x[1 \dots n]$ consistent with the questions the sorting algorithm has asked.

Initially, there are no edges, and any ordering of the vertices of the DAG is a topological sort. When the sorting algorithm is done, they'll just be a single topological sort possible. The DAG will just be one long chain of vertices.

So, the goal of the DAG adversary should be to ensure the number of topological sortings of the DAG is maximized after its answer. Looking at our pictures above, the "Yes" DAG has $5! = 120$ topological sortings, whereas the "No" DAG, has only $2 \times 4! = 48$ topological sortings. Our "count the descendants" heuristic payed off. But why the heuristic, why not just count?

11 Counting is Hard

So, we have a simple new adversary right? When it gets asked a question, it sees if adding the "Yes edge" or "No edge" gives a DAG with a larger number of topological sortings, and answers accordingly.

Unfortunately, a little Googling convinces us we'd end up with an amazingly slow adversary. Counting topological sortings isn't just NP-complete, it's #P-complete (`http://dl.acm.org/citation.cfm?id=103441`).

What about an approximation? More Googling. That too, is rather slow. Even approximately counting (`http://arxiv.org/pdf/1010.4981.pdf`) takes (roughly speaking) $O(n^3(\log L(P))^2)$ time, here $L(P)$ is the number of topological sortings of the DAG (or, linear extensions of the partial order). In our case, early on, since the DAG has no edges, we'll have $L(P) = n!$, and so the time just to answer one question (i.e. compare two elements!) will be, again ignoring messy terms, $O(n^5)$.

12 Our DAG is Slow

Even with the simple count-the-descendants heuristic. It's worth coming to terms with the fact that our DAG adversary is sadly quite slow. A simple implementation (`https://github.com/nicknash/AdversaryExpe`
of the underlying DAG is an adjacency list representation. This will take $\Theta(1)$ time to add a new a edge, $\Theta(n)$ time to query for a directed path, and $\Theta(n)$ time to count descendants.

This is pretty bad, especially if we're driving a sorting algorithm into quadratic territory – our program will need cubic time! Perhaps worse, the DAG will have a quadratic number of edges in this case.

13 Tweaking the DAG

There's scarily extensive literature on maintaining transitive closures (or reductions) of DAGs — probably relevant if we want to optimize our implementation. Rather than do all that reading though, we'll just attack the constant factors to get an implementation that's fast enough to be of use.

It turns out just one idea gives about a factor of 2 speed-up: Our adversary is designed so that the queries for directed paths should almost always not find any path at all (or if you prefer, in the worst case, this is what must happen). This means our path queries will explore all the descendants of their source node. Aaaand, one more thing: we only ever count descendants (`https://github.com/nicknash/AdversaryExperiments/blob/master/Adversaries/DAGAdversary.cs`) having performed a path query.

With that little idea in hand, we can implement a DAG (`https://github.com/nicknash/AdversaryExperiments/b`
that's only guaranteed to report descendant counts correctly if we've just performed a path query — which we'll force to always explore all descendants. This should save about half the work of the adversary. You'll be excited to know I've tested it and it does.

One or two other uninteresting tweaks give us about a 1.5x speed-up: Removing some pretty Linq and avoiding repeatedly clearing and initializing traversal data by working in "epochs".

14 Who's the Baddest?

So, is there any use for this DAG adversary? Well, kind of.

A while ago, I wanted to stress test some selection algorithms. These are quicksort-like algorithms that, given some whole number k, then re-arrange an array A such that $A[k]$ has everything less than it to its left, and everything greater to its right.

The details of the algorithms don't matter too much. They're based on Floyd and Rivest's SELECT algorithm (`http://dl.acm.org/citation.cfm?id=360694`). Also, they're variations on "introspective" algorithms. That is, they start by using one algorithm, and if they notice that algorithm approaching its worst case time, they switch to an algorithm with better worst case time. They do this because although the algorithm they switch to is better in the worst case, it's much slower on average. The idea is to get the good average case performance of say, quicksort, with the worst case guarantee of heapsort.

In the case of these selection algorithms, the algorithm they switch to is labelled "Fallback" (red-line). Here's a plot of their performance against McIlroy's quicksort killer:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/07/intro_selects_mcilroy.png`

Link: `http://nicknash.me/wp-content/uploads/2012/07/intro_selects_mcilroy.png`

The plot above should be a bit worrying. Both the Intro-SELECT and Hybrid-SELECT algorithm switch to the Fallback when they hit their primary algorithm's worst case. They spend some time realizing things are going badly, so in those cases, they perform worse than just running the Fallback in the first place. Except that, in the plot above, Hybrid-SELECT is doing better than the Fallback: It *hasn't* encountered its worst case, despite the efforts of McIlroy's quicksort killer. Meanwhile, Intro-SELECT has hit its worst case – it's taking longer than the Fallback.

The next plot shows what happens against the DAG adversary:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/07/intro_selects_dag_adv.png`

Link: `http://nicknash.me/wp-content/uploads/2012/07/intro_selects_dag_adv.png`

Here, we can see what we expect (well, except for that weird dip for Hybrid-SELECT – I haven't looked into that!) the Fallback is fastest, and the two other algorithms perform worse. The DAG adversary has forced them to switch to the Fallback after some wasted work.

So, in this rather unusual situation, the DAG adversary is more powerful than McIlroy's quicksort killer. Of course, it's also much slower, but at least we're getting something in return for the time and CPU heat.

15 Code

I've implemented the adversaries described here in C#. I'm sure you're just itching to take a look:

https://github.com/nicknash/AdversaryExperiments (`https://github.com/nicknash/AdversaryExperiments`)